

Rakib SHEIKH
Travaux pratiques : Big Data
rakib.sheikh@cyu.fr

TP 1 : Stack Hadoop

Pré-requis :

- Docker
- Java 11 Eclipse Temurin (via sdkman si possible) <https://sdkman.io/>

Si jamais vous êtes sur Windows : <https://kontext.tech/project/hadoop/article/install-hadoop-330-on-windows-10-step-by-step-guide>

Partie 1 : Hadoop HDFS

Exercice 1

Nous allons utiliser docker pour déployer une infrastructure Hadoop. Cela vous évitera de perdre du temps consacré à l'installation de celle-ci qui peut s'avérer fastidieuse.

1. Commencez par créer un fichier s'appelant `docker-compose.yml`

Si vous avez bien suivi le cours, notre objectif est de déployer les composants suivants :

- Un NameNode
- Un DataNode

2. Pour ce faire, nous allons devoir remplir le `docker-compose.yml`. Le langage utilisé est YAML
 - Commencez par rédiger en première ligne `services:`

Le service nous sert à déclarer l'ensemble des services qui vont constituer notre cluster HDFS, il y aura donc 2 services à déclarer à savoir le NameNode et le DataNode.

- Déclarez le service `namenode:` (N'oubliez pas la tabulation pour indenter)

Maintenant, nous allons renseigner les informations suivantes :

- `image: apache/hadoop:3.4.0 -->` Cette ligne correspond à l'image que nous souhaitons récupérer depuis <https://hub.docker.com/r/apache/hadoop/tags>. Le 3.4.0 correspond à la version à utiliser.
- `container_name: namenode -->` Ici nous précisons le nom du container qui sera namenode
- `user: root -->` Ici c'est une spécificité de l'image, nous devons être root pour pouvoir lancer les commandes que nous allons faire exécuter dès le lancement du container
- `environment:` `-->` Nous permet de fixer les variables d'environnement pour notre container.
- `- HADOOP_HOME=/opt/hadoop -->` Cela nous permet de fixer où se situent les installations de Hadoop dans le container.
- Nous allons ouvrir le port 9870 pour que l'on puisse y accéder depuis l'extérieur du container docker grâce à notre navigateur internet plus tard dans cette section

```
ports:
```

```
yml
```

```
- "9870:9870"
```

- `command: ["/bin/bash", "/start-hdfs.sh"] -->` Cette commande permet de lancer les commandes dès le lancement du container. Ici, à l'aide du shell sh, nous lancerons le fichier `start-hdfs.sh` que nous rédigerons plus tard.
- Nous allons garder en trace les dossiers que peut créer notre container. Nous allons donc mapper les répertoires du container par rapport au Host. Vous allez devoir créer physiquement trois dossiers à savoir :

- `hadoop_namenode`
- `hadoop_config` (nous y reviendrons plus tard)
- Le fichier `start-hdfs.sh` (nous y reviendrons plus tard)

```
volumes:
  - ./hadoop_namenode:/opt/hadoop/data/nameNode
  - ./hadoop_config:/opt/hadoop/etc/hadoop
  - ./shared:/shared
  - ./start-hdfs.sh:/start-hdfs.sh
```

- Nous allons fixer une adresse IP à notre container

```
networks:
  hdfs_network:
    ipv4_address: 172.20.0.2
```

- Et en fin de fichier, nous allons créer la carte réseau (Nous sortons de la section `services`)

```
networks:
  hdfs_network:
    ipam:
      driver: default
      config:
        - subnet: 172.20.0.0/16
```

- Faire la même chose pour le service `datanode` mais avec les informations suivantes :

- `container_name` : `datanode1`
- Ne pas ajouter le paramètre `user`.
- Le volume

```
volumes:
  - ./hadoop_datanode1:/opt/hadoop/data/dataNode
  - ./hadoop_config:/opt/hadoop/etc/hadoop
  - ./init-datanode.sh:/init-datanode.sh
```

- `command` : `["/bin/bash", "/init-datanode.sh"]`
- `ipv4_address` : `172.20.0.3`
- `ports` : `"9864:9864"`

- Nous allons maintenant configurer les services interne du Namenode et du Datanode

- Récupérez le fichier `core-site.xml` et `hdfs-site.xml` à partir de ce lien <https://gist.github.com/Noobzik/ddfc792c56ae11bcb9b546c0bd0fad3c>
- Stockez ces fichiers dans le dossier `hadoop_config`
- Modifiez les valeurs suivantes :

- Nous allons finir la configuration par la création des fichiers :

- `init-datanode.sh`

```
#!/bin/bash
rm -rf /opt/hadoop/data/dataNode/*
chown -R hadoop:hadoop /opt/hadoop/data/dataNode
chmod 755 /opt/hadoop/data/dataNode
```

```
hdfs datanode
```

- `start_hdfs.sh`

```
#!/bin/bash
bash

if [ ! -d "/opt/hadoop/data/nameNode/current" ]; then
    echo "Formatting NameNode..."
    hdfs namenode -format -nonInteractive
fi
hdfs namenode
```

6. Moment de vérité : tapez la commande `docker compose up` sur un terminal ouvert à l'emplacement du fichier `docker-compose.yml`
- Vérifiez le bon lancement en vous connectant avec un navigateur internet à l'adresse `http://localhost:9870`
 - Mise en garde : L'image pèse 1.18GB, assurez-vous d'avoir une bonne bande passante, ou utilisez le partage de connexion de votre téléphone portable si le réseau de l'école est saturé.
 - Remarque 2 : En cas d'erreur lié au network

```
failed to create network bigdata_cytech_hdfs_network: Error response from daemon: invalid pool request: Pool overlaps with other one on this address space
```

Changez l'ensemble de l'IP en un chiffre supérieur, (par exemple 172.60 au lieu de 172.20), cela est dû par la présence d'un autre container docker avec une carte réseau active mais non détruite.

Voici les commandes de Hadoop les plus utilisés

Instruction	Fonctionnalité
<code>hdfs dfs -ls</code>	Afficher le contenu du répertoire racine
<code>hdfs dfs -put file.txt</code>	Upload un fichier dans hadoop (à partir du répertoire courant de votre disque local)
<code>hdfs dfs -get file.txt</code>	Download un fichier à partir de hadoop sur votre disque local
<code>hdfs dfs -tail file.txt</code>	Lire les dernières lignes du fichier
<code>hdfs dfs -cat file.txt</code>	Affiche tout le contenu du fichier
<code>hdfs dfs -mv file.txt newfile.txt</code>	Renommer (ou déplacer) le fichier
<code>hdfs dfs -rm newfile.txt</code>	Supprimer le fichier
<code>hdfs dfs -mkdir myinput</code>	Créer un répertoire

Remarque, vous devez préciser toujours la racine sinon, vous allez avoir une erreur de type

```
ls: `.`: No such file or directory
```

Pour pouvoir utiliser les commandes de Hadoop HDFS, nous devons nous connecter à notre container namenode.

```
docker exec -it namenode /bin/bash
bash
```

Exercice 2 : Mes premières commandes HDFS

Le but de cette exercice est de stocker des fichiers, de les lire en deux parties.

Les prépaîtres

1. Téléchargez le fichier `purchase.txt.gz` à cette adresse. : <https://github.com/CodeMangler/udacity-hadoop-course/blob/ec6bbb839bdc6e701f802c523497fef4e1c206d0/Datasets/purchases.txt.gz>
2. Décompressez le fichier gz à l'aide de la commande suivante : `gzip --decompress purchases.txt.gz`
3. Déplacez le fichier `purchases.txt` dans le dossier `shared` que vous avez du créer à l'exercice 1.

Le début réel de l'exercice

À l'aide des commandes hadoop.

1. Connectez-vous à votre container `namenode` et créez un dossier `input` sous la forme d'un HDFS

```
hdfs dfs -mkdir -p input
```

2. Maintenant, copiez le contenu du dossier `/shared` (Local) vers `input` (HDFS)
3. Listez le contenu du `input` (HDFS)
4. Maintenant affichez les dernières lignes de `purchase.txt` à l'aide de `tail` depuis le `input` du HDFS. (Elle prends deux argument, le source et la destination).
5. Allez sur l'interface graphique situé à `localhost:9870` et trouvez le fichier `purchase.txt`

Partie 2 : YARN et MapReduce

Nous allons devoir modifier notre `docker-compose` afin de pouvoir incorporer le service YARN. Et nous l'utilisons pour executer un simple programme de MapReduce à partir de ce service.

Exercice 1 : Modification du `docker-compose.yml`

Nous allons devoir modifier notre `docker-compose.yml` afin de pouvoir y ajouter un nouveau service YARN, qui nous permettra de lancer un algorithme MapReduce sur un WordCount en Java.

1. Tapez la commande `docker compose down` pour éteindre le cluster
2. Nous allons maintenant vous fournir le `docker-compose.yml` voici le github : https://github.com/Noobzik/big_data_ing3_cytech
3. Relancez le cluster avec `docker-compose.yml`
4. Testez l'interface YARN avec l'adresse suivante : `http://localhost:8088/cluster`

Exercice 2 : Application simple d'un MapReduce

Cette exercice est adapté depuis la version officiel de Apache Hadoop sur WordCount.

Cette exercice sera divisé en deux section, à savoir faire fonctionner MapReduce en local et sur le cluster docker que nous avons crée.

Section 1 : En locale.

1. Si ce n'est déjà fait, clonez le répertoire suivant : https://github.com/Noobzik/big_data_ing3_cytech/
- **TokenizerMapper** : Ce fichier contient la méthode `map`. L'objectif de cette méthode est d'énumérer les mots de chaque ligne. Chaque mot trouvé aura pour valeur 1. Elle permet de préparer à `reduce` qui va réaliser le calcul du nombre de fois que le mot apparaît. Le résultat sera envoyé à la méthode `reduce` de la classe `IntSumReducer`.
 - **IntSumReducer** : Ce fichier contient la méthode `reduce`. L'objectif de cette méthode est de regrouper les mots en comptant le nombre de fois que ce mot apparaît. Elle réutilise le résultat généré par `map` de la classe `TokenizerMapper`.
 - **WordCount** : C'est notre main ! Elle permet de construire le cheminement de notre algorithme MapReduce. Comme nous allons compiler le projet sous la forme d'un `jar`, nous allons fixer la classe `WordCount` en tant que classe `Main`, la classe `TokenizerMapper` en tant que classe de `Mapper` et `IntSumReduce` en tant que classe de `Reducer`. Nous préciserons également le type des clés et de la valeur qui sera `Text` et

`IntWritable` respectivement. Et enfin nous définissons une entrée et une sortie de fichier, que nous donnons en entrée de programme et récupéré depuis les `args` du programme.

2. Exécutez le programme avec le dossier input contenant le fichier `purchases.txt` en premier paramètre, ainsi que le nom d'un dossier non créée (par exemple output) dont nous allons stocker les résultats.
 - Que remarquez-vous ?
 - Inspectez le fichier `part-r-00000` et que remarquez-vous ?

Section 2 : En cluster

Maintenant, nous allons faire la même chose mais à l'aide d'un cluster hadoop que nous avons créé à l'aide de Docker. C'est comme si vous allez soumettre un code vers une base de données, dont l'exécution sera réalisé sur place, et non l'inverse (c'est à dire récupérer les données pour appliquer une transformation pour ensuite renvoyer le résultat vers une base de données distant.)

1. Compilez le projet afin que cela forme un fichier jar en incluant les dépendances nécessaires. (Nous l'appellerons `wc.jar`).
 - Sous IntelliJ, il faut ouvrir le paramètre Maven qui se situe à droite de l'écran, puis sur `wordcount Lifecycle` et cliquer sur `package`. Un fichier jar sera généré dans le dossier target.
2. Placez le fichier avec les dépendance dans votre répertoire du container, pour cela, utilisez la commande : `docker cp .\wordcount-1.0-SNAPSHOT-jar-with-dependencies.jar resourcemanager:/root/wc.jar`
 - Pour information, nous sommes dans le container de YARN, c'est à dire le `resourcemanager`. La commande pour accéder à docker interactive est la suivante : `docker exec -it resourcemanager /bin/bash`
3. Exécutez le programme à l'aide de la commande suivante :

```
yarn jar /root/wc.jar /user/root/input /user/root/output
```

bash

3. Inspectez le fichier de sortie `output/part-r-00000` à l'aide de la commande `hadoop fs -cat`
 - Que remarquez-vous ?

Remarque : Il est possible de suivre l'avancement du job mais il n'est pas disponible sur ce TP.

Si tout s'est bien passé, voici le résultat que vous devriez avoir dans le fichier `part-r-00000` (fin de fichier)

```
Women's 230050
Worth 40336
York 40364
and 229667
```

log

Partie 3 : La transition du futur : Minio

Actuellement, Hadoop HDFS est un framework qui est malheureusement populaire et ancien. Il est très difficile de réaliser sa maintenance et c'est l'une des raison pour lesquelles les entreprise envisage la migration vers le Cloud.

Mais la plupart d'entre eux ne peuvent pas se permettre de réaliser la migration vers le cloud en raison de la réglementation en vigueur de leur secteur d'activité (notamment la finance).

Il existe donc une solution moderne qui se calque de l'utilisation d'un service en particulier d'Amazon S3 (« Un hadoop mais du cloud ») disponible en open-source. C'est Minio.

Exercice 1 : Création du `docker-compose.yml`

1. Nous allons créer un nouveau fichier `docker-compose.yml` sur un répertoire différent pour cette partie. Ajoutez les informations suivante.

```
services:
```

yaml

```
minio:
  hostname: minio
  image: "minio/minio"
  container_name: minio
  ports:
    - "9001:9001"
    - "9000:9000"
  command: [ "server", "/data", "--console-address", ":9001" ]
  volumes:
    - ./minio:/data
  environment:
    - MINIO_ROOT_USER=minio
    - MINIO_ROOT_PASSWORD=minio123
```

2. Vérifiez la bonne mise en place en se connectant à l'interface graphique situé à `localhost:9000`
3. Installez la CLI de Minio <https://min.io/docs/minio/linux/reference/minio-mc.html>
4. Faites les configuration nécessaire pour que la connexion via la CLI fonctionne.
5. Créez un bucket appelé `test` et insérez les fichiers `file1.txt` et `purchase.txt` à la racine, à l'aide de la CLI `mc`. (Rappel, le `file.txt` à été crée lors de Hadoop)

Exercice 2 : Maintenant via Python

Il est possible de faire exactement la même chose mais avec du code Python. L'avantage c'est que Minio réutilise le code de AWS S3 pour pouvoir communiquer avec Minio Server.

1. Écrivez un programme permettant de mettre le fichier `purchase.txt` dans un bucket nommé `python` en utilisant les commandes AWS.

Partie 4 : Le futur nous a trahit ! RustFS à la rescousse

Reproduire la partie 3 de minio, mais cette fois-ci avec RustFS

Et ça sera la fin du TP, ouf !